Final Projects 2026/27

Speaker: Jeremy Miller (Shamoon College of Engineering (SCE))

Workshop

14th June 2026

The succintness gap of Quantum Finite Automaton

Difference between DFA and QFA

The goal of this project is to solve the succinctness problem in quantum formal language theory by investigating the 1-Qubit Quantum Finite Automaton (QFA) through the lens of linear algebraic transformations. While QFAs are known to require exponentially fewer states than classical deterministic automata for certain languages, finding the exact bounds of this succinctness gap remains a critical open challenge. By conceptualizing quantum operations as continuous state transitions driven by 2×2 unitary matrices, we establish a rigorous mathematical framework to analyze quantum state complexity. We demonstrate the step-by-step processing of binary strings to calculate probability amplitudes, utilizing this matrix-driven approach as a foundational step toward systematically characterizing the exponential memory efficiency of quantum automata.

The succintness gap of Quantum Finite Automaton

Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A DFA exists in only one state at a time.
- A QFA exists in a **superposition of states**.
- i.e. a QFA exists in all of its states at the same time.
- A non-deterministic QFA can recognize non-regular languages.

Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A DFA exists in only one state at a time.
- A QFA exists in a **superposition of states**.
- i.e. a QFA exists in all of its states at the same time.
- A non-deterministic QFA can recognize non-regular languages.

Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A DFA exists in only one state at a time.
- A QFA exists in a **superposition of states**.
- i.e. a QFA exists in all of its states at the same time.
- A non-deterministic QFA can recognize non-regular languages.

Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A DFA exists in only one state at a time.
- A QFA exists in a **superposition of states**.
- i.e. a QFA exists in all of its states at the same time.
- A non-deterministic QFA can recognize non-regular languages.

Difference between DFA and QFA

- A **Quantum Finite Automaton (QFA)** is a computational model analogous to a DFA including quantum computing phenomena.
- A DFA exists in only one state at a time.
- A QFA exists in a **superposition of states**.
- i.e. a QFA exists in all of its states at the same time.
- A non-deterministic QFA can recognize non-regular languages.

Definition of a QFA

- A Measure-Once QFA is defined:

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a \in \Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle \psi_1 | \psi_2 \rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a\in\Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle\psi_1|\psi_2\rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- A Measure-Once QFA is defined:

$$M = \left(Q, \Sigma, |\psi_0\rangle, \{U_a\}_{a\in\Sigma}, F\right)$$

- $Q \in H$ is a set of **states** represented as an **orthonormal basis** of a **Hilbert space**, H.
- A **Hilbert space** is a vector space with a defined inner-product $\langle\psi_1|\psi_2\rangle$ for all $\psi_1, \psi_2 \in H$.
- Σ is a finite alphabet.
- $|\psi_0\rangle \in H$ is a **unit vector** in H that defined the **initial state**.
- For every letter $a \in \Sigma$, U_a is a **unitary matrix** that dictates how the QFA calculates letters, analgous to the transition function of DFA.
- $F \in H$ is the set of **accepting states**.

Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator U_{σ_i} acts on each state vector to evolve the QFA to the next state vector:

- **Acceptance:**The probablity that the QFA accepts w is the quared norm of the projection of the final state onto the accepting subspace:

Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator U_{σ_i} acts on each state vector to evolve the QFA to the next state vector:

$$|\psi_w\rangle = U_{\sigma_n} U_{\sigma_{n-1}} \cdots U_{\sigma_1} |\psi_0\rangle \ .$$

- **Acceptance:** The probablity that the QFA accepts w is the quared norm of the projection of the final state onto the accepting subspace:

$$P\left(w \in L(M)\right) = \sum_{\psi \in F} \langle \psi | \psi \rangle \ .$$

Definition of a QFA

- **Evolution**: On each input $w = \sigma_1 \cdots \sigma_n$, a unitary operator U_{σ_i} acts on each state vector to evolve the QFA to the next state vector:

$$|\psi_w\rangle = U_{\sigma_n} U_{\sigma_{n-1}} \cdots U_{\sigma_1} |\psi_0\rangle \ .$$

- **Acceptance:**The probablity that the QFA accepts w is the quared norm of the projection of the final state onto the accepting subspace:

$$P\left(w \in L(M)\right) = \sum_{\psi \in F} \langle \psi | \psi \rangle \ .$$

Problem 1

- **Problem 1:** For any language L accepted by a DFA, A, does there always exist an equivalent QFA, B that accepts L?
-

$$\forall\, A \in DFA:\ L = L(A) \qquad \exists\, B \in QFA:\ L = L(B) \qquad ?$$

$$L(D) = L(Q)\ ?$$

Problem 1

- **Problem 1:** For any language L accepted by a DFA, A, does there always exist an equivalent QFA, B that accepts L?
-

$$\forall\, A \in DFA:\ L = L(A) \qquad \exists\, B \in QFA:\ L = L(B) \qquad ?$$

$$L(D) = L(Q)\ ?$$

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Problem 2

- Let L be a language such that there exists a DFA,

$$A = (Q_A, \Sigma, \delta, q_0, F)$$

and a QFA,

$$B = \left(Q_B, \Sigma, |\psi_0\rangle, (U_a)_{a\in\Sigma}, P_{\text{acc}}\right)$$

such that

$$L = L(A) = L(B) \ .$$

- Let A be the minimum DFA that accepts L, and B be the minimum QFA that accepts L.
- The succintness gap $\theta(L)$ is the difference between the minimum number of states:

$$\theta(L) = |Q_A| - |Q_B| \ .$$

- **Problem 2:**
  1. For a given language L, calculate the minimal QFA, B such that $L = L(B)$.
  2. Build an algorithm to calculate the succinctness gap $\theta(L)$ for a specific language.
  3. Find a formula for $\theta(L)$ for any language L.

Turing machine model of DNA decryption

Turing machine model of DNA decryption

This project implements a Turing machine that computes a biological function: translating a DNA sequence over the alphabet $\{A, T, C, G\}$ into a corresponding protein or phenotypic characteristic. The input is a gene encoded as a finite DNA string, and the output is either a named protein (e.g., insulin, haemoglobin) or a trait determined by that gene. The alphabet $\{A, T, C, G\}$ represents the four nucleotide bases—adenine, thymine, cytosine, and guanine—that form the base pairs of the DNA double helix. The tape of a Turing machine closely resembles a single DNA strand, while a two-tape Turing machine mirrors the structure of the DNA double helix. The machine simulates transcription and translation by mapping codons to amino acids and applying rules of gene expression.

Turing machine model of DNA decryption

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- Deoxyribo Nucleic Acid (DNA) is a double sequence of four different connected molecules:
  1. Adenine (A),
  2. Thymine (T),
  3. Cytosine (C),
  4. Guanine (G).
-

A	T	G	G	C	T	A	C	G	T	A	A	C	G	G

T	A	C	C	G	A	T	G	C	A	T	T	G	C	C

Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

A	T	G	G	C	T	A	C	G	T	A	A

T	A	C	C	G	A	T	G	C	A	T	T

- **RNA Sequence (12 Base Singlets)**

A	T	G	G	C	T	A	C	G	T	A	A

- **RNA Sequence with Amino Acids**

A	T	G	G	C	T	A	C	G	T	A	A
Met			Ala			Thr			Stop		

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

A	T	G	G	C	T	A	C	G	T	A	A

T	A	C	C	G	A	T	G	C	A	T	T

- **RNA Sequence (12 Base Singlets)**

A	T	G	G	C	T	A	C	G	T	A	A

- **RNA Sequence with Amino Acids**

A	T	G	G	C	T	A	C	G	T	A	A
Met			Ala			Thr			Stop		

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

A	T	G	G	C	T	A	C	G	T	A	A

T	A	C	C	G	A	T	G	C	A	T	T

- **RNA Sequence (12 Base Singlets)**

A	T	G	G	C	T	A	C	G	T	A	A

- **RNA Sequence with Amino Acids**

A	T	G	G	C	T	A	C	G	T	A	A
Met			Ala			Thr			Stop		

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

Turing machine model of DNA decryption

- **DNA Sequence (12 Base Pairs)**

A	T	G	G	C	T	A	C	G	T	A	A

T	A	C	C	G	A	T	G	C	A	T	T

- **RNA Sequence (12 Base Singlets)**

A	T	G	G	C	T	A	C	G	T	A	A

- **RNA Sequence with Amino Acids**

A	T	G	G	C	T	A	C	G	T	A	A
Met			Ala			Thr			Stop		

- Tyr – Arg – Cys – Ile $\Longrightarrow$ Haemoglobin

Trading Eye: Optimizing Stock-Market Trading Strategies

Trading Eye: Optimizing Stock-Market Trading Strategies

The goal of this project is to construct *Trading Eye*: an algorithmic trading platform tailored to design, backtest, and optimize real-world financial strategies. It scans financial markets for the first ten or more assets—stocks or currencies—that meet a defined long-entry signal. The user is free to build customized strategies based on the basic indicators including Simple, Exponential, and Hull Moving Averages (SMAs, EMAs, HMAs), Volatility, Bollinger bands, and more. Trading Eye should also accommodate delta-neutral portfolios based on the Black-Scholes equation. The app simulates and visualizes profit margins in real time or using historical data. It supports multiple strategies, each based on probabilistic models trained on past market behavior, and allocates investments using weighted distributions. Users can compare the performance of arbitrary strategies via clear, interactive graphs. The system is designed to aid traders in identifying high-probability opportunities and optimizing portfolio decisions with data-driven insights.

Quantum computing methods of improving time and space-complexity

Quantum computing methods of improving time and space-complexity

The goal of this project is to check if the time and space complexity of computationally hard problems is genuinely improved by a quantum computer compared to classical systems. The project will specifically focus on evaluating a list of problems including the Traveling Salesperson Problem (TSP), Max-Cut, and Circuit-SAT. Quantum computers utilize qubits, which can exist in multiple states simultaneously due to the phenomena of superposition and entanglement. By exploring these parallel processing capabilities, we will design and implement quantum optimization algorithms (such as QAOA) to rigorously measure algorithmic performance and demonstrate potential practical quantum advantages in both time and space complexity.

A combinatoric model for genetic mutation

A combinatoric model for genetic mutation

The goal of this project is to develop a computational model that calculates the probability of hereditary diseases based on complex gene combinations and specific mutations. Genetic predispositions are often polygenic, meaning multiple genes interact to influence a single phenotype or overall disease risk. By applying probabilistic models and combinatorial algorithms to genomic datasets, we can systematically map allele frequencies and interactions to accurately predict the statistical likelihood of specific genetic disorders.

Quantum Entanglement methods for mapping key distributions

Quantum Entanglement methods for mapping key distributions

The goal of this project is to model and simulate quantum entanglement to develop highly secure communication protocols focusing on quantum key-mapping and key distribution (QKD). Entanglement is a purely quantum phenomenon where the states of two or more particles become fundamentally correlated, enabling the secure generation and mapping of cryptographic keys across vast distances. This non-local physical property acts as the absolute cornerstone for advanced quantum key-mapping algorithms, ensuring that any external eavesdropping immediately disrupts the quantum state and is detected.

Shor's algorithm for decrypting RSA ciphertext in the absence of keys

Shor's algorithm for decrypting RSA ciphertext in the absence of keys

The goal of this project is to implement and analyze Shor's algorithm for factoring large integers, demonstrating the theoretical vulnerability of the RSA cryptosystem to quantum computing attacks. The security of RSA historically relies on the computational difficulty of prime factorization for classical computers. Shor's algorithm, however, cleverly leverages quantum Fourier transforms and periodicity finding to solve integer factorization in polynomial time, posing a critical threat to modern cryptographic paradigms.

Stock market strategies based on the Black-Scholls equation and market reversal

Stock market strategies based on the Black-Scholls equation and market reversal

The goal of this project is to design robust algorithmic trading strategies by integrating the Black-Scholes model with statistical methods for detecting random market reversals. The Black-Scholes mathematical model estimates the theoretical value of put and call options based on dynamic variables like volatility and time to expiration. Combining this continuous-time finance model with stochastic calculus allows us to model random walks and construct delta neutral trading portfolios, successfully identifying probabilistic mean-reversion anomalies in asset prices for optimized trading.

Combinatorial calculation of the probability of gene mutation

Combinatorial calculation of the probability of gene mutation

The goal of this project is to develop a computational model that calculates the probability of hereditary diseases based on complex gene combinations and specific mutations. Genetic predispositions are often polygenic, meaning multiple genes interact to influence a single phenotype or overall disease risk. By applying probabilistic models and combinatorial algorithms to genomic datasets, we can systematically map allele frequencies and interactions to accurately predict the statistical likelihood of specific genetic disorders.

A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.

A numerical algorithm for partial differentiatial systems with an complexity improvement of one order of time.

The goal of this project is to implement and optimize numerical algorithms for solving coupled systems of ordinary and partial differential equations while rigorously calculating their time and space complexity. Many physical and biological systems are modeled using continuous differential equations lacking closed-form solutions. By focusing on the Runge-Kutta method and other discretization techniques, we aim to approximate the dynamic behavior of these systems with high algorithmic precision. The project will specifically analyze the improved efficiency of optimized Runge-Kutta variations and benchmark their memory usage and execution time.

Building a Large Language Model to solve analytical equations.

Building a Large Language Model to solve analytical equations.

The goal of this project is to construct a scaled-down Large Language Model (LLM) to rigorously explore the linear algebra and deep mathematical foundations of modern artificial intelligence. By building neural networks from the ground up, the project will investigate non-linear transformations driven by the sigmoid model and other continuous activation functions. The architecture's underlying mathematics relies heavily on high-dimensional linear algebra, specifically focusing on complex matrix multiplications, matrix inversions for weight space analysis, and gradient descent optimization during backpropagation. This mathematical approach illuminates how dense vector embeddings and probabilistic token predictions fundamentally emerge from pure algebraic operations.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As the
was some unprocessed data that should have been added to the final
page this extra page has been added to receive it.
If you rerun the document (without altering it) this surplus page will
away, because LaTeX now knows how many pages to expect for this
document.